# COMPUTER VISION

# LABSHEET 7

NAME – HARSH RAJ

CUID – CU240251762

SEC – C, ROLL- 23

```
#NAME - PALLAV PANKAJ
#ROLL-40



import cv2
import numpy as np
import os

# EXPERIMENT 7
# MOTION ESTIMATION USING OPTICAL FLOW

VIDEO_PATH = "input.mp4"

print("=" * 60)
print("       OPTICAL FLOW - EXPERIMENT 7")
print("=" * 60)

# ------------------------------------------------------------
# Check video file
# ------------------------------------------------------------

if not os.path.isfile(VIDEO_PATH):
    print("\nERROR: input.mp4 was not found!")
    print("Put input.mp4 in the same folder as exp7.py")
    input("\nPress Enter to exit...")
    exit()

print("\nVideo found:", VIDEO_PATH)

# ------------------------------------------------------------
# Open video
# ------------------------------------------------------------

cap = cv2.VideoCapture(VIDEO_PATH)

if not cap.isOpened():
    print("\nERROR: OpenCV could not open the video.")
    input("\nPress Enter to exit...")
```

```
    exit()

# Get video information
fps = cap.get(cv2.CAP_PROP_FPS)
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print("\nVideo Information")
print("--------------------------")
print("Width       :", width)
print("Height      :", height)
print("FPS         :", fps)
print("Total frames:", total_frames)

# ------------------------------------------------------------
# Read first frame
# ------------------------------------------------------------

ret, old_frame = cap.read()

if not ret:
    print("\nERROR: Could not read video frame.")
    cap.release()
    input("\nPress Enter to exit...")
    exit()

print("\nVideo successfully loaded.")
print("Starting optical flow...\n")

# Convert first frame to grayscale
old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)

# ------------------------------------------------------------
# Detect feature points
# ------------------------------------------------------------

feature_params = dict(
    maxCorners=200,
    qualityLevel=0.2,
    minDistance=5,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
```

```
    old_gray,
    mask=None,
    **feature_params
)

if p0 is None:
    print("WARNING: No feature points detected.")
else:
    print("Initial feature points:", len(p0))
#NAME - PALLAV PANKAJ
#ROLL-40
# ---------------------------------------------------------------
# Lucas-Kanade parameters
# ---------------------------------------------------------------

lk_params = dict(
    winSize=(21, 21),
    maxLevel=3,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        30,
        0.01
    )
)
#NAME - PALLAV PANKAJ
#ROLL-40
# ---------------------------------------------------------------
# Drawing mask
# ---------------------------------------------------------------

trajectory_mask = np.zeros_like(old_frame)

# ---------------------------------------------------------------
# Create window
# ---------------------------------------------------------------

window_name = "Experiment 7 - Optical Flow Comparison"

cv2.namedWindow(
    window_name,
    cv2.WINDOW_NORMAL
)

# 2 panels x 600 pixels
cv2.resizeWindow(
    window_name,
    1200,
```

```
    500
)
#NAME - PALLAV PANKAJ
#ROLL-40
# ------------------------------------------------------------
# Frame counter
# ------------------------------------------------------------

frame_number = 1

# ============================================================
# MAIN LOOP
# ============================================================

while True:

    ret, frame = cap.read()

    if not ret:
        print("\nVideo reached the end.")
        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # ========================================================
    # LUCAS-KANADE SPARSE OPTICAL FLOW
    # ========================================================

    lk_result = frame.copy()

    if p0 is not None and len(p0) > 0:

        p1, status, error = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )

        if p1 is not None:

            good_new = p1[status == 1]
            good_old = p0[status == 1]
```

```
            for new, old in zip(
                good_new,
                good_old
            ):

                new_x, new_y = new.ravel()
                old_x, old_y = old.ravel()

                new_x = int(new_x)
                new_y = int(new_y)
                old_x = int(old_x)
                old_y = int(old_y)

                # Draw trajectory
                trajectory_mask = cv2.line(
                    trajectory_mask,
                    (new_x, new_y),
                    (old_x, old_y),
                    (0, 255, 0),
                    2
                )

                # Draw point
                lk_result = cv2.circle(
                    lk_result,
                    (new_x, new_y),
                    4,
                    (0, 0, 255),
                    -1
                )

                # Draw arrow
                cv2.arrowedLine(
                    lk_result,
                    (old_x, old_y),
                    (new_x, new_y),
                    (255, 0, 0),
                    2,
                    tipLength=0.3
                )

            # Add trajectories
            lk_result = cv2.add(
                lk_result,
                trajectory_mask
            )

            # Update points
```

```
            p0 = good_new.reshape(
                -1,
                1,
                2
            )

        else:

            p0 = cv2.goodFeaturesToTrack(
                frame_gray,
                mask=None,
                **feature_params
            )

            trajectory_mask = np.zeros_like(frame)

    else:

        # Detect features again
        p0 = cv2.goodFeaturesToTrack(
            frame_gray,
            mask=None,
            **feature_params
        )

        trajectory_mask = np.zeros_like(frame)
#NAME - PALLAV PANKAJ
#ROLL-40
    # ============================================================
    # FARNEBACK DENSE OPTICAL FLOW
    # ============================================================

    flow = cv2.calcOpticalFlowFarneback(
        old_gray,
        frame_gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    # Magnitude and direction
    magnitude, angle = cv2.cartToPolar(
        flow[..., 0],
```

```
        flow[..., 1]
    )
#NAME - PALLAV PANKAJ
#ROLL-40
    # ========================================================
    # COLOR-CODED FLOW
    # ========================================================

    hsv = np.zeros_like(frame)

    hsv[..., 1] = 255

    hsv[..., 0] = (
        angle * 180 / np.pi / 2
    ).astype(np.uint8)

    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)

    farneback_result = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    # ========================================================
    # RESIZE OUTPUTS
    # ========================================================

    panel_width = 600
    panel_height = 450

    left = cv2.resize(
        lk_result,
        (panel_width, panel_height)
    )

    right = cv2.resize(
        farneback_result,
        (panel_width, panel_height)
    )

    # ========================================================
    # LABELS
```

```
    # ============================================================

    cv2.putText(
        left,
        "LUCAS-KANADE - SPARSE",
        (15, 35),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 255, 255),
        2
    )

    cv2.putText(
        right,
        "FARNEBACK - DENSE",
        (15, 35),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 255, 255),
        2
    )

    # Frame number
    cv2.putText(
        left,
        "Frame: " + str(frame_number),
        (15, 75),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (255, 255, 255),
        2
    )
#NAME - PALLAV PANKAJ
#ROLL-40
    # ============================================================
    # SIDE-BY-SIDE
    # ============================================================

    combined = np.hstack(
        (left, right)
    )

    # ============================================================
    # SHOW OUTPUT
    # ============================================================

    cv2.imshow(
        window_name,
```

```
        combined
    )

    # ============================================================
    # UPDATE
    # ============================================================

    old_gray = frame_gray.copy()

    frame_number += 1

    # ------------------------------------------------------------
    # Keyboard
    # ------------------------------------------------------------

    key = cv2.waitKey(30) & 0xFF

    if key == ord("q") or key == 27:
        print("\nProgram stopped by user.")
        break

    # Press R to reset feature points
    if key == ord("r"):

        p0 = cv2.goodFeaturesToTrack(
            old_gray,
            mask=None,
            **feature_params
        )

        trajectory_mask = np.zeros_like(frame)

        print("Feature points reset.")

# ============================================================
# END
# ============================================================

cap.release()

print("\nOptical flow processing completed.")

# Keep final output window visible
print("Press any key in the output window to close it.")

cv2.waitKey(0)

cv2.destroyAllWindows()
```

```
#NAME - PALLAV PANKAJ
#ROLL-40

# Questions and Answers
# 1. What is Optical Flow? How is it used in computer vision?

# Optical Flow is the apparent motion of objects, surfaces, or the camera between two consecutive frames of a video.

# It calculates the displacement of pixels or feature points from one frame to another.

# It is used for:

# Object tracking
# Motion detection
# Video surveillance
# Autonomous vehicles
# Gesture recognition
# Sports analysis
# Action recognition

# Optical flow is particularly useful for understanding dynamic scenes and motion patterns.

# 2. Explain the working principle of the Lucas-Kanade Optical Flow algorithm.

# Lucas-Kanade is a Sparse Optical Flow technique.

# Its working can be summarized as:

# Detect important feature points in the first frame.
# Convert consecutive frames to grayscale.
# Search for the corresponding feature points in the next frame.
# Estimate the displacement of these points.
# Use a small neighborhood around each feature to calculate motion.
# Draw the motion vectors or trajectories.

# The basic optical-flow constraint is:

# $$ I_xu + I_yv + I_t = 0 $$

# where:

# \(I_x\) = image gradient in x direction
# \(I_y\) = image gradient in y direction
# \(I_t\) = temporal gradient
```

```
# \(u,v\) = velocity components

# The algorithm assumes that nearby pixels have approximately similar motion.

# 3. What is Dense Optical Flow? How does it differ from Sparse Optical Flow?

# Dense Optical Flow calculates motion for almost every pixel in the image.

# Sparse Optical Flow calculates motion only for selected feature points.

# Feature   Sparse Optical Flow Dense Optical Flow
# Motion points Selected features   Almost every pixel
# Example   Lucas-Kanade    Farneback
# Computational cost    Lower   Higher
# Output    Motion vectors at features  Motion field over image
# Suitable for  Object tracking Complete motion analysis

# The experiment specifically requires Lucas-Kanade for sparse flow and
Farneback for dense flow.

# 4. Compare the Lucas-Kanade and Farneback optical flow algorithms.
# Parameter Lucas-Kanade    Farneback
# Type  Sparse  Dense
# Motion estimation Selected feature points Every pixel
# Speed Relatively fast More computationally expensive
# Output    Point trajectories  Dense motion field
# Memory requirement    Lower   Higher
# Best suited for   Feature/object tracking Detailed scene motion
# Visualization Arrows/trajectories Color-coded flow
# Robustness    Good for trackable features Good for overall motion estimation

# The lab sheet asks comparison based on accuracy, computational complexity,
and robustness.

# 5. What assumptions are made while computing optical flow?

# Common assumptions are:

# Brightness constancy – the brightness of a moving point remains
approximately constant.
# Small motion – displacement between consecutive frames is relatively small.
# Spatial coherence – neighboring pixels generally have similar motion.
# The image contains sufficient texture or features for motion estimation.
# The motion between frames changes smoothly in local regions.

# These assumptions allow the algorithms to estimate motion from consecutive
frames.
```

```
# 6. What factors can affect the accuracy of optical flow estimation?

# Several factors can reduce accuracy:

# Object speed
# Poor lighting
# Sudden illumination changes
# Camera movement
# Motion blur
# Occlusion
# Low-texture regions
# Large displacement between frames
# Noise
# Rapid changes in object appearance

# The experiment specifically asks students to analyze object speed, lighting conditions, and camera motion.

# 7. Mention five real-world applications of optical flow.

# Five applications are:

# Autonomous driving – detecting movement of vehicles and pedestrians.
# Video surveillance – detecting unusual or suspicious movement.
# Object tracking – following objects across video frames.
# Gesture recognition – interpreting hand and body movements.
# Sports analytics – analyzing player and ball movement.

# Other applications include robotics and augmented reality.

# 8. Why are grayscale images generally used for optical flow computation?

# Grayscale images are generally used because optical flow primarily relies on intensity changes and image gradients.

# Using grayscale:

# Reduces computational complexity.
# Reduces the amount of data processed.
# Simplifies gradient calculations.
# Makes motion estimation faster.
# Avoids separately processing three color channels.

# Therefore, converting consecutive frames to grayscale is a standard preprocessing step for optical flow.

# The experiment instructions explicitly specify converting consecutive frames to grayscale before optical-flow computation.
```

```
# 9. What are the limitations of optical flow algorithms in real-world
environments?

# Major limitations include:

# Sensitive to lighting changes.
# Difficulty with very fast motion.
# Difficulty with large displacement.
# Occlusion can cause incorrect tracking.
# Textureless regions provide little information.
# Camera motion can affect the estimated object motion.
# Motion blur can reduce accuracy.
# Noise can introduce incorrect motion vectors.
# Dense methods can require significant computational resources.

# Thus, real-world environments can make accurate motion estimation
challenging.

# 10. How does optical flow contribute to autonomous driving, video
surveillance, and action recognition?

# Autonomous driving:
# Optical flow helps estimate the movement of nearby vehicles, pedestrians,
and environmental features. This can contribute to motion understanding and
navigation.

# Video surveillance:
# It can identify moving objects and help detect unusual motion patterns,
making it useful for surveillance systems.

# Action recognition:
# Human movement produces characteristic optical-flow patterns. These patterns
can be used to recognize actions such as walking, running, waving, or jumping.

# Therefore, optical flow provides important motion information for dynamic
scene understanding.
```

OUTPUT

```
Video found: input.mp4


Video Information

----------------------------

Width      : 1920
```

Height : 1080

FPS : 24.0

Total frames: 671

Video successfully loaded.

Starting optical flow...

Initial feature points: 163

Program stopped by user.

Optical flow processing completed.

Press any key in the output window to close it